

Architettura

Datapath

Performance determinate da 3 fattori:

- 1- Numero di istruzioni
- 2- Ciclo di clock
- 3- Ciclo di clock per istruzione (CPI)

Tre tipi di istruzioni:

- 1- R-Type
- 2- I-Type
- 3- Branches

Datapath singolo ciclo:

Ogni istruzione va in esecuzione su un fronte del clock e termina sul fronte di clock successivo.

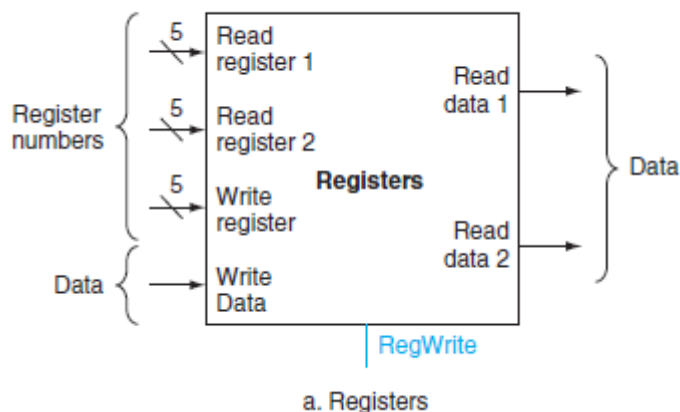
Questo approccio non è pratico, è più lento di una implementazione che consente a differenti classi di istruzioni di usare un differente numero di cicli di clock, ognuno di essi potrebbe essere più breve.

Tutti i segnali devono essere propagati dallo state element 1, attraverso i circuiti combinatori, allo state element 2 nel tempo di un ciclo di clock, il tempo necessario al segnale per raggiungere il secondo elemento di memorizzazione definisce la lunghezza del ciclo di clock.

Con il metodo **edge-triggered** si può leggere e scrivere (uno state element) nello stesso ciclo di clock senza creare problemi.

Per creare un **datapath** abbiamo bisogno di alcuni elementi:

- 1- **Program Counter (PC)**: usato per memorizzare l'indirizzo dell'istruzione corrente, dopo il prelevamento dell'indirizzo necessario nella fase di fetch dell'istruzione il PC viene incrementato di 4 per puntare la prossima istruzione.
- 2- **Register File**: Una collezione di registri in cui è possibile leggere o scrivere ogni registro specificando il numero del registro interessato. Il register file contiene inoltre il registro di stato della macchina. Il register file è strutturato come segue:



Gli ingressi per la selezione dei registri hanno ampiezza di 5 bit per consentire l'indirizzamento di $2^5 = 32$ Registri.

Mentre i bus relativi ai dati hanno ampiezza di 32 bit.

- 3- **ALU**: L'ALU possiede 2 ingressi da 32 bit ciascuno e produce un output da 32 bit, e un output di 1 bit se il risultato è 0.

È dotata inoltre di un bus da 4 bit per il controllo delle sue funzioni (AluOp)

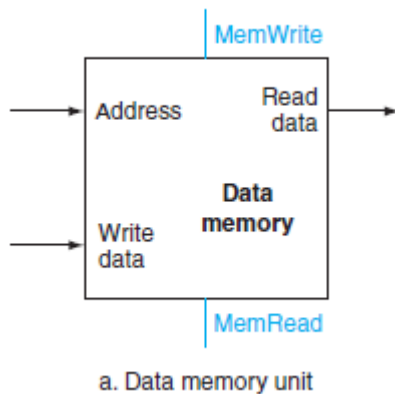
Considerando le istruzioni SW e LW (sw/lw \$t1, offset_value(\$t2)) queste istruzioni calcolano l'indirizzo di memoria sommando il registro base (\$t2) ai 16 bit di offset contenuti nell'istruzione.

Abbiamo bisogno di una unità di estensione di segno per estendere i 16bit di offset nei 32bit richiesti.

- 4- **Sign-Extend** : Ripete il bit più significativo dei 16bit di offset fino ad ottenere 32 bit

Inoltre necessitiamo di una memoria da cui leggere o in cui scrivere. La memoria verrà scritta nelle istruzioni di store e letta nelle istruzioni di load, essa è strutturata come segue:

- 5- **Memoria:**



Per le istruzioni di branch (es. beq) abbiamo bisogno di calcolare il branch target address relativo all'istruzione, per fare ciò sommiamo il campo offset esteso di segno con il PC prestando attenzione a 2 particolari:

- 1- L'indirizzo base per la branch è l'indirizzo dell'istruzione successiva alla branch, quindi il PC è PC+4
- 2- Deve esserci un allineamento alla word, quindi dopo aver esteso il segno bisogna shiftare di 2 bit a sinistra l'indirizzo prima di sommarlo al PC.

Dunque il datapath deve eseguire 2 operazioni, calcolare il branch target address e comparare il contenuto dei due registri, Se il risultato dell'uscita zero dell'Alu è alto, allora la Branch ha dato esito positivo.

Per l'istruzione di salto il target address è calcolato come segue:

- 1- Prendo i 26 bit del campo indirizzo dell'istruzione
- 2- Li shifto di 2 bit verso sinistra
- 3- Li concateno ai 4 bit più significativi del PC

Nell'architettura MIPS le branch sono ritardate (si, come te), questo significa che la istruzione immediatamente successiva alla branch è comunque eseguita indipendentemente dal risultato delle branch, per semplicità nella parte non-eludibile le branch non verranno considerate ritardate, ma voi si.

The Alu control:

Possiamo generare un input di 4 bit usando una piccola control-unit che ha come input il campo function dell'istruzione e 2 bit di controllo che chiameremo ALUOp, ALUOp indica quale operazione deve effettuare l'ALU:

- 00 Esegue una somma
- 01 Esegue una sottrazione

- 10 Esegue quello che è indicato dal campo funzione

Instruction opcode	ALUOp	Instruction operation	Funct field	Desired ALU action	ALU control input
LW	00	load word	XXXXXX	add	0010
SW	00	store word	XXXXXX	add	0010
Branch equal	01	branch equal	XXXXXX	subtract	0110
R-type	10	add	100000	add	0010
R-type	10	subtract	100010	subtract	0110
R-type	10	AND	100100	and	0000
R-type	10	OR	100101	or	0001
R-type	10	set on less than	101010	set on less than	0111

FIGURE 5.12 How the ALU control bits are set depends on the ALUOp control bits and the different function codes for the R-type instruction. The opcode, listed in the first column, determines the setting of the ALUOp bits. All the encodings are shown in binary. Notice that when the ALUOp code is 00 or 01, the desired ALU action does not depend on the function code field; in this case, we say that we “don’t care” about the value of the function code, and the funct field is shown as XXXXXX. When the ALUOp value is 10, then the function code is used to set the ALU control input.

Bisogna fare **delle osservazioni riguardanti i campi dell’istruzione:**

- Il campo Opcode, è sempre contenuto nei bit 31:26, noi ci riferiremo a questi bit come Op[5:0]
- I 2 registri da leggere sono sempre identificati dai campi rs e rt, alle posizioni 25:21 e 20:16, questo vale per le istruzioni R-Type, Beq e per la sw.
- Il registro base per la lw e sw è sempre nei bit in posizione 25:21(rs).
- I 16 bit di offset per la beq, lw e sw sono sempre in posizione 15:0.
- Il registro di destinazione può essere in uno di 2 posti, per la lw in 20:16(rt), mentre per un R-Type è in posizione 15:11(rd), aggiungeremo quindi un multiplexor per selezionare il campo in base all’istruzione.

Perché una implementazione a singolo ciclo non è usata oggi?

Essa funziona correttamente ma è inefficiente, per capire ciò bisogna notare che il ciclo di clock deve avere la stessa lunghezza per ogni operazione nell’implementazione a singolo ciclo e il CPI deve essere 1, e il ciclo di clock è determinato dal più lungo circuito della macchina, questo circuito è sicuramente quello della lw, che usa 5 unità funzionali in serie, quindi le performance non sono molto buone perché molte classi di istruzioni potrebbero essere eseguite in un ciclo di clock più corto.

IMPLEMENTAZIONE MULTI-CICLO

In una implementazione **multiciclo** ogni step di esecuzione impiega un ciclo di clock, e permette alle unità funzionali di essere usate per più di una istruzione, questa condivisione aiuta a ridurre l’ammontare di richiesta di hardware.

La capacità di consentire alle istruzioni di usare un numero differente di cicli di clock e la abilità di condividere parti funzionali durante l’esecuzione di una singola istruzione sono i maggiori vantaggi dell’implementazione multiciclo.

Al termine di ogni singolo ciclo tutti i dati usati nei cicli di clock successivi vanno memorizzati, nella nostra implementazione assumiamo che in un ciclo di clock possono rientrare al massimo una di queste operazioni:

- Un accesso a memoria
- Un accesso al register file (2 letture o una scrittura)
- Una operazione dell’ALU

Alla fine del ciclo ogni output prodotto da una di queste operazioni deve essere salvato in un registro temporaneo.

I seguenti registri sono stati aggiunti per questo motivo:

- **Instruction Register (IR)** e il **Memory data register(MDR)** per salvare rispettivamente l’istruzione letta dalla memoria e la lettura di un dato, mantiene l’istruzione in memoria fino alla fine dell’esecuzione dell’istruzione.

- I registri **A** e **B** sono usati per mantenere il valore degli operandi letti dal register file.
- Il registro **ALUOut** salva il risultato dell'ALU.

Per le istruzioni di branch e salto ci **sono 3 possibili sorgenti** del valore da scrivere nel PC:

- 1- L'uscita dell'ALU che è il PC+4, questo valore va memorizzato direttamente nel PC.
- 2- Il registro ALUOut che è dove memorizzeremo l'indirizzo calcolato per la branch
- 3- I 26 bit meno significativi del IR shiftati di 2 verso sinistra e concatenati con i 4 bit più significativi del PC+4, questa è la sorgente per le istruzioni di jump.

Useremo quindi 2 controlli per modificare il valore del PC, **PCWrite** che causa una scrittura incondizionata del PC, e **PCWr_Cond** che causa una scrittura del PC solo nel caso la condizione risulti vera.

Questi 2 controlli vanno collegati al **PC write control signal**, per determinare quando il PC deve essere scritto durante una branch mettiamo in una AND l'uscita **Zero** dell'ALU e con il **PCWr_Cond**, il risultato di questa AND viene messo in una OR con il **PC write control signal**.

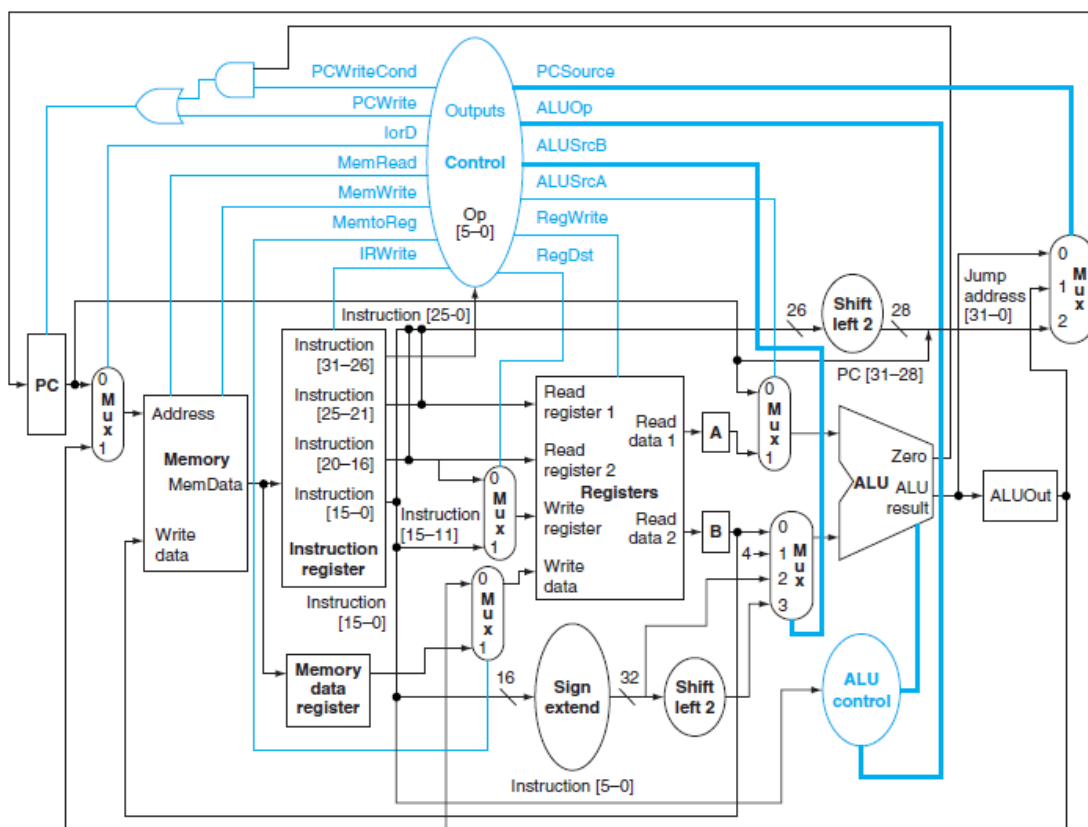


FIGURE 5.28 The complete datapath for the multicycle implementation together with the necessary control lines. The con-

Actions of the 1-bit control signals

Signal name	Effect when deasserted	Effect when asserted
RegDst	The register file destination number for the Write register comes from the rt field.	The register file destination number for the Write register comes from the rd field.
RegWrite	None.	The general-purpose register selected by the Write register number is written with the value of the Write data input.
ALUSrcA	The first ALU operand is the PC.	The first ALU operand comes from the A register.
MemRead	None.	Content of memory at the location specified by the Address input is put on Memory data output.
MemWrite	None.	Memory contents at the location specified by the Address input is replaced by value on Write data input.
MemtoReg	The value fed to the register file Write data input comes from ALUOut.	The value fed to the register file Write data input comes from the MDR.
lorD	The PC is used to supply the address to the memory unit.	ALUOut is used to supply the address to the memory unit.
IRWrite	None.	The output of the memory is written into the IR.
PCWrite	None.	The PC is written; the source is controlled by PCSource.
PCWriteCond	None.	The PC is written if the Zero output from the ALU is also active.

Actions of the 2-bit control signals

Signal name	Value (binary)	Effect
ALUOp	00	The ALU performs an add operation.
	01	The ALU performs a subtract operation.
	10	The funct field of the instruction determines the ALU operation.
ALUSrcB	00	The second input to the ALU comes from the B register.
	01	The second input to the ALU is the constant 4.
	10	The second input to the ALU is the sign-extended, lower 16 bits of the IR.
	11	The second input to the ALU is the sign-extended, lower 16 bits of the IR shifted left 2 bits.
PCSource	00	Output of the ALU (PC + 4) is sent to the PC for writing.
	01	The contents of ALUOut (the branch target address) are sent to the PC for writing.
	10	The jump target address (IR[25:0] shifted left 2 bits and concatenated with PC + 4[31:28]) is sent to the PC for writing.

FIGURE 5.29 The action caused by the setting of each control signal in Figure 5.28 on page 323. The top table describes the 1-bit control signals, while the bottom table describes the 2-bit signals. Only those control lines that affect multiplexors have an action when they are deasserted. This information is similar to that in Figure 5.16 on page 306 for the single-cycle datapath, but adds several new control lines (IRWrite, PCWrite, PCWriteCond, ALUSrcB, and PCSource) and removes control lines that are no longer used or have been replaced (PCSrc, Branch, and Jump).

ANALISI DI UN'ISTRUZIONE NEL DATAPATH

Step 1

- Fetch (lettura)

L'istruzione viene caricata in memoria e la ALU esegue il calcolo dell'indirizzo dell'istruzione seguente

IR (Registro istruzione) <= Memoria[PC]

PC = PC + 4

- Decode (decodifica)

Viene letta l'istruzione precedente, letti i registri indicati da rs e rt dell'istruzione e viene computato dall'ALU l'eventuale target address in caso di branch.

A <= IR[25-21] (è uguale a rs)

B <= IR[20-16] (è uguale a rt)

ALUOut <= PC + shift<<2(estensione(IR[15-0])) → SOLO NEL CASO DI BRANCH

- Execute (esecuzione)

Il ciclo di esecuzione dipende dal tipo di istruzione caricata che può essere di tipo “Riferimento di memoria” (lw,sw), “Operazione aritmetico-logica” (add,sub,and,or,slt), “Branch” (beq), “jump” (j)

1. Riferimento di memoria

L'ALU somma gli operandi per creare l'indirizzo di memoria (rs+offset)

ALUOut <= A + estensione(IR[15:0])

2. Operazione aritmetico-logica

L'ALU compie l'operazione richiesta in base all'OPCode passato.

ALUOut <= A operazione B

3. Branch

L'ALU compara i due registri letti nella decode tramite una sottrazione e se si equivalgono, cioè il risultato della sottrazione è 0, lo compara con il segnale Zero e setta il segnale di controllo del salto condizionato a 1

SE (A==B) PC<=ALUOut

4. Jump

Il PC viene direttamente aggiornato con la concatenazione dei suoi primi 4 bit e dello shift di 2bit a sinistra del target scritto nella jump

PC <= PC[31:28] || shift<<2(IR[25:0])

- Accesso alla memoria

Finita l'elaborazione dell'istruzione, i risultati vengono scritti nella memoria o, in caso di lettura, nel memory data register per essere utilizzati al prossimo giro di clock (nel caso di istruzioni di load)

Memoria[ALUOut] <= B oppure MDR<=Memoria[ALUOut]

Nel caso di operazioni aritmetico-logiche, il risultato viene scritto nel registro destinazione

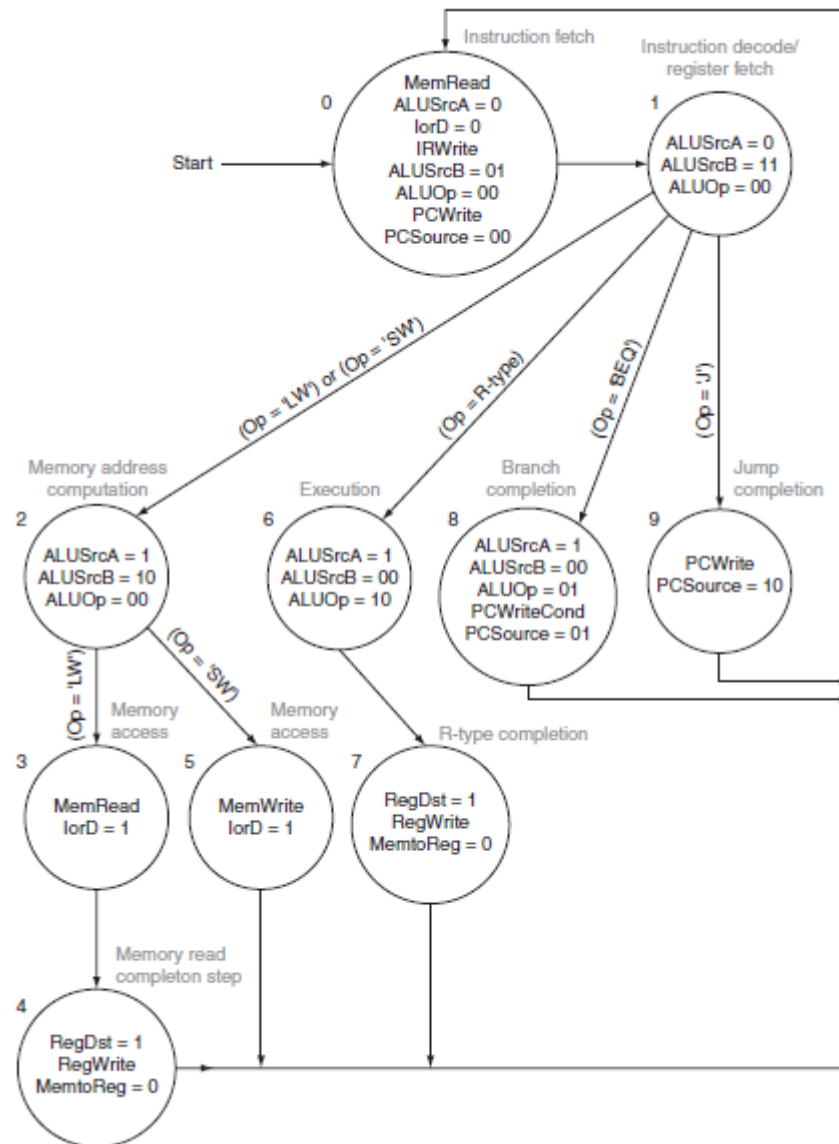
Rd[IR[15:11]] <=ALUOut

- Completamento lettura dalla memoria

Questo step viene realizzato solo nel caso di operazioni di riferimento di memoria, in particolare nelle operazioni di Load e consiste nel caricamento del registro indicato dall'istruzione con il dato letto dalla memoria

Rd[IR[20:16]] <= MDR

FINITE STATE MACHINE



Eccezioni

Fanno scattare una sequenza di istruzioni prestabilita

- Eccezioni

Impreviste dal programmatore (overflow, word non allineate, opcode non esistente), hanno luogo in sincrono con il programma

- Traps

Sincrone ma Previste (Syscall) il programmatore utilizza il meccanismo delle eccezioni, fa accadere un evento fuori dal flusso normale

- Interrupt

Asincrono(non sono legati all'esecuzione delle istruzioni) e imprevisto, es. pressione di un tasto dall'utente
Ma viene percepito in modo sincrono.

Alterazione del flusso delle eccezioni è diverso da quello che usiamo per le procedure

Nel gestore si scoprirà chi ha generato il problema e come gestirlo

La gestione è software, i registri utilizzati sono \$K0 e \$K1(riservati al kernel), il gestore salva lo stato della computazione interrotta, perché dopo bisognerà riprenderla, se ha bisogno di più registri oltre a k0 e k1, allora salva in memoria dove a nessuno da fastidio (sw in una struttura di tipo stack).

Le convenzioni adottate per i registri interi del MIPS sono riassunti di seguito:

Nome	Numero	Utilizzo	Preservato durante le chiamate
\$zero	0	costante zero	Riservato MIPS
\$at	1	riservato per l'assemblatore	Riservato Compiler
\$v0-\$v1	2-3	valori di ritorno di una procedura	No
\$a0-\$a3	4-7	argomenti di una procedura	No
\$t0-\$t7	8-15	registri temporanei (non salvati)	No
\$s0-\$s7	16-23	registri salvati	Si
\$t8-\$t9	24-25	registri temporanei (non salvati)	No
\$k0-\$k1	26-27	gestione delle eccezioni	Riservato OS
\$gp	28	puntatore alla global area (dati)	Si
\$sp	29	stack pointer	Si
\$s8	30	registro salvato (fp)	Si
\$ra	31	indirizzo di ritorno	No

Di seguito è riportata la suddivisione della memoria usata nel MIPS32. Lo Stack e lo Heap condividono l'area dati lasciata libera dai dati statici.

0x1000.0000-0x7fff.ffff	Area Dati	Stack ↓
		Heap ↑
		Dati statici
0x0040.0000-0x0fff.ffff	Area Testo	
0x0000.0000-0x003f.ffff	Area O.S.	

Successivamente al salvataggio dello stato bisogna Identificare la causa:

Per fare questo introduciamo nel circuito il **registro Cause**

Il registro cause ha in ingresso l'uscita di un multiplexer(controllato da IntCause) che ha 2 entrate: 0 se l'errore è Opcode non esistente e 1 se si tratta di Overflow.

il selettore del multiplexer è IntCause(1 bit), il valore di Cause è scritto solo quando necessario dal controllo CauseWrite.

Dopo aver gestito (se è stato possibile) l'eccezione devo ritornare al programma interrotto: Fase di Restore, si ritorna con la istruzione ERET.

Ma dove torno? Mi serve l'indirizzo di ritorno, un registro apposito per contenere il vecchio PC abbandonato → EPC
Il **registro EPC** è controllato da EPCWrite, di norma non ci vogliamo scrivere, l'uscita dell'ALU è l'ingresso dell'EPC

Al verificarsi di una eccezione Devo sapere l'indirizzo della prima istruzione del gestore e metterla nel PC, modifico il multiplexer del PCsrc, il suo ingresso 11 viene posto fisso a 0x8000180 (registro di partenza dei gestori)

Quando esco dallo stato 1(Della macchina a stati finiti) il PC è già a + 4, allora sottraggo al PC 4 e lo metto nell'EPC, e modifico il PCSrc = 11, e così torno torno allo stato 0 (quello di Fetch).

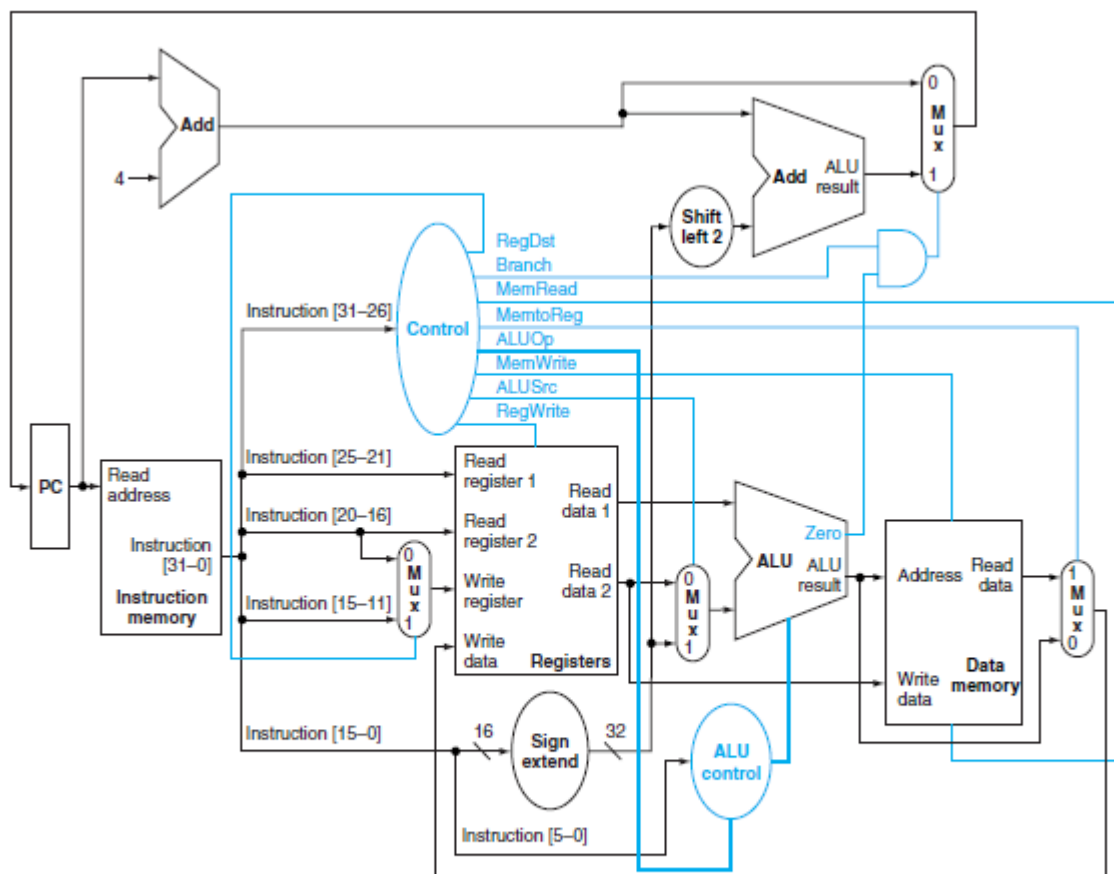


FIGURE 5.17 The simple datapath with the control unit. The input to the control unit is the 6-bit opcode field from the instruction. The outputs of the control unit consist of three 1-bit signals that are used to control multiplexers (RegDst, ALUSrc, and MemtoReg), three signals for controlling reads and writes in the register file and data memory (RegWrite, MemRead, and MemWrite), a 1-bit signal used in determining whether to possibly branch (Branch), and a 2-bit control signal for the ALU (ALUOp). An AND gate is used to combine the branch control signal and the Zero output from the ALU; the AND gate output controls the selection of the next PC. Notice that PCSrc is now a derived signal, rather than one coming from the instruction.

Un metodo per la gestione delle interruzioni è la vettorizzazione, nella **vettorizzazione** l'indirizzo a cui è trasferito il controllo è determinato dalla causa dell'eccezione:

ES.

Tipo dell'eccezione	Indirizzo vettore delle eccezioni
Istruzione indefinita	0xC000 0000
Overflow	0xC000 0020

L'O.S. conosce la ragione dell'eccezione dall'indirizzo al quale è inizializzato, gli indirizzi sono separati da 32 Byte o 4 istruzioni, e l'O.S. deve registrare la ragione dell'eccezione e magari eseguire qualche operazione, se le eccezioni non sono vettorizzate un singolo punto di ingresso può essere utilizzato per tutte le eccezioni, e l'O.S. identifica la causa decodificando il registro di stato.

Per determinare la causa dell'eccezione il gestore delle eccezioni (Usando assembly si parla di Exception.s) osserva i registri del **co-processore 0** relativi alla gestione delle eccezioni.

Registro	Numero	Significato
BadVAddr	8	L'indirizzo di memoria a cui ci si riferisce a causato un'eccezione
Statuts	12	Maschera delle interruzioni e bit di abilitazione

Cause	13	Tipo di eccezione e interrupt in attesa di servizio
EPC	14	Indirizzo dell'istruzione problematica

Per prelevare i dati dai registri del c0 si usano le seguenti istr:

- mtc0 – move to c0
- mfc0 – move from c0

Registro Cause (32 bit)

6:2 - Codice eccezione

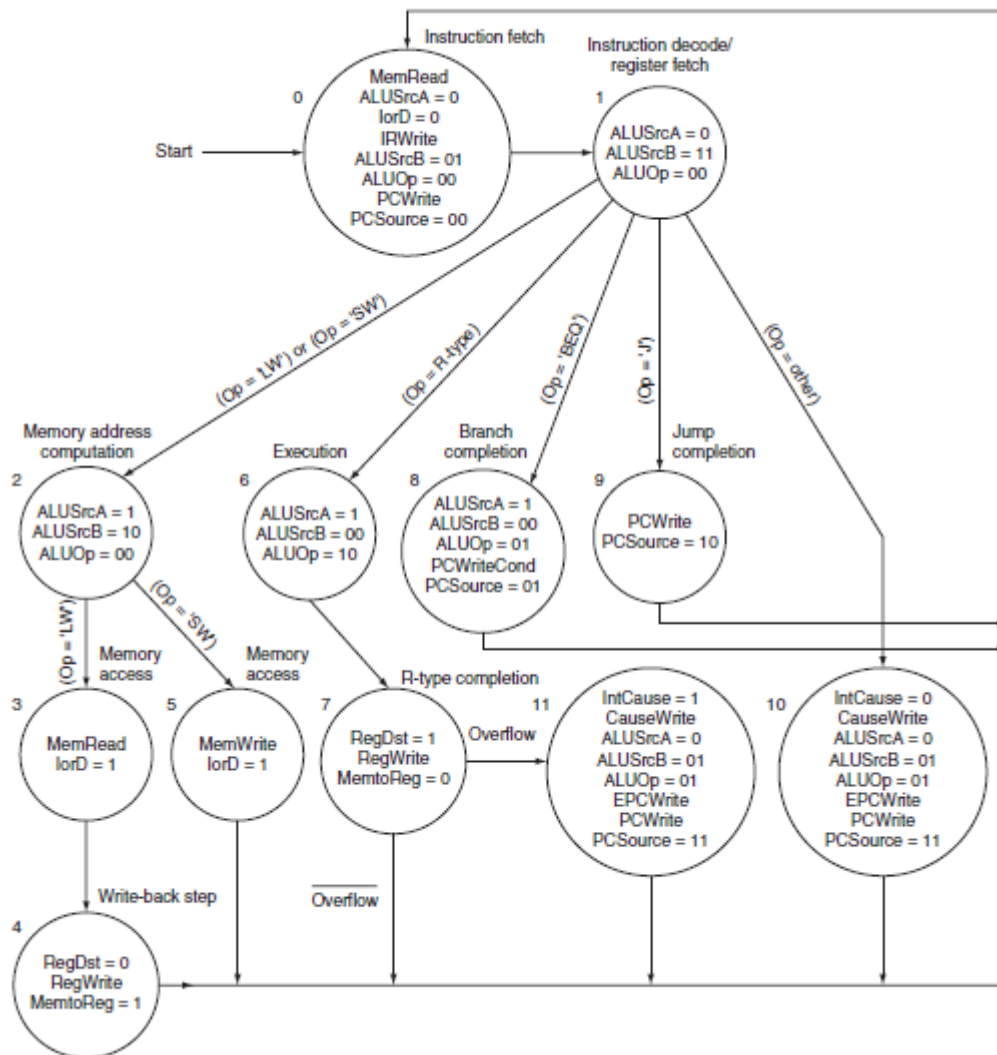
15:8 - Interrupt pendenti

31:30 - Branch delay

Registro Status

- interrupt enable [0:0]
 - 1- interruzioni sono abilitate
 - 0- interruzioni disabilitate
- Exception level [1:1]
 - 1- Settato a 1 dopo che l'eccezione è sollevata
 - 0- Il valore di default
- User mode [4:4]
 - 1- Se il processore è in user mode
 - 0- Se il processore è in kernel mode
- Interrupt mask esprime gli interrupt abilitati [15:8]

MACCHINA A STATI FINITI + EXC



Periferiche

Componente elettromeccanico (Es. stampante)

La CPU comunica con le periferiche attraverso una interfaccia costituita dai registri periferica, la periferica interessata è specificata tramite un indirizzo.

Bus

- Dati
- Controllo
- Indirizzi

I/O memory mapped

La parte di memoria oltre la locazione 0x7ffffffc riservata per il sistema viene utilizzata anche per la comunicazione con le periferiche.

Verso l'esterno la periferica si interfaccia con i registri periferica, uno di stato e uno di dati.

Il registro di stato rappresenta lo stato della periferica, che viene letto dalla CPU, lo stato contiene informazioni riguardanti la prontezza della periferica per una trasmissione di dati o la disponibilità dei dati richiesti per il trasferimento.

Il registro dati è di input o output per la CPU a seconda del tipo di periferica.

Vediamo ora in che modo la CPU può controllare lo stato della periferica, il problema si pone quando la CPU esegue istruzioni con una certa frequenza e la periferica non genera dati alla stessa frequenza della CPU.

La tecnica più semplice è quella a controllo di programma:

La CPU controlla il valore del registro di stato della periferica finché non risulta pronta, la CPU deve ripetere l'operazione di controllo del valore, la CPU resterà impegnata a ripetere questa operazione il che comporta ad uno spreco di energia e non permette alla CPU di eseguire altre istruzioni, questa attesa svolta eseguendo le istruzioni del ciclo si chiama **Busy Wait**.

Nel momento in cui la periferica diventerà pronta la CPU potrà proseguire con il trasferimento. Il trasferimento si ottiene caricando prima il valore nei registri della CPU e poi con una lw/sw in memoria.

Nello spazio di memoria limitato inferiormente dall'indirizzo precedente vengono dedicate porzioni di memoria per i registri delle varie periferiche, **memory mapped** significa che ogni registro della periferica appare anche in memoria in una locazione ad esso dedicata.

Dal momento che i registri della periferica sono mappati in memoria, le operazioni di accesso alla periferica saranno simili a quelli già visti per la memoria del sistema.

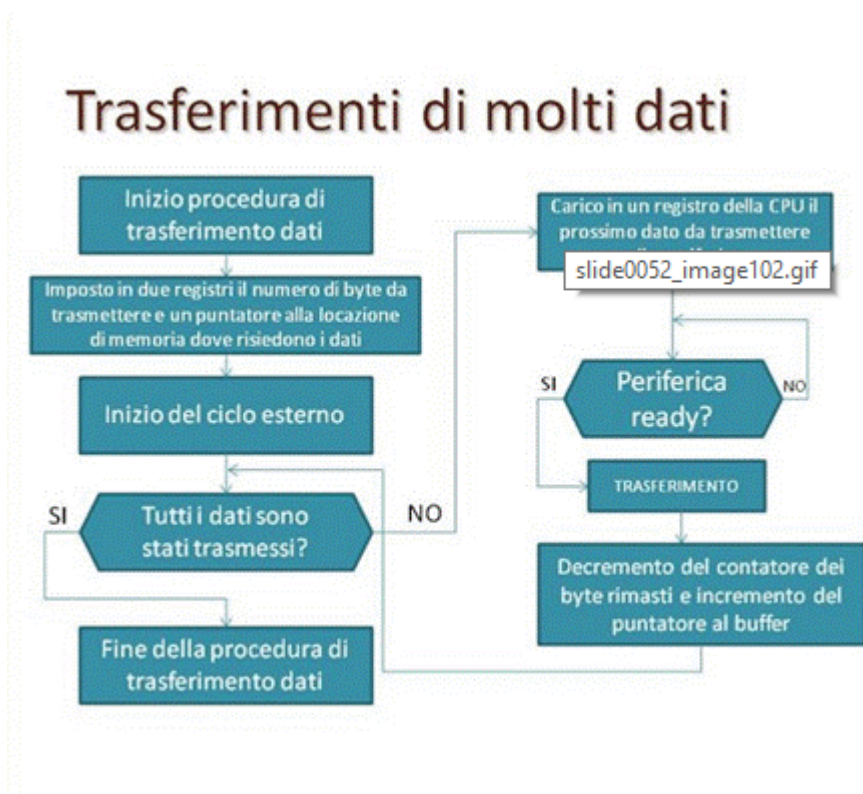
Non tutte le architetture prevedono il collegamento diretto tra periferiche e memoria, come ad esempio quella Intel.

L'interfaccia ha una banda passante cioè la quantità di dati che si può trasferire per unità di tempo. È quindi una misura di flusso.

La latenza è il tempo che intercorre tra il momento in cui la periferica è pronta e il momento in cui il dato viene trasferito, è quindi una misura di tempo.

Nel caso a controllo di programma la latenza è minima e la banda sarà ampia.

Es. trasferimento dati con Busy Wait



Interrupt

Vogliamo liberare la CPU dal Busy Wait, serve quindi un meccanismo della sincronizzazione tra la CPU e le periferiche senza che la CPU sia impegnata.

Questo meccanismo si chiama Interruzioni e prevede una complicazione hardware.

Sul fronte del bus e delle periferiche bisogna aggiungere un bus di controllo per trasmettere l'informazione di pronto alla CPU, questa linea del bus è chiamata "linea di richiesta".

Quando la periferica genera un'interrupt la CPU deve eseguire una serie di istruzioni prestabilite contenute a partire da una locazione di memoria nota a priori.

Queste operazioni sono la save e la restore che abbiamo già visto per le eccezioni.

All'arrivo dell'interrupt tutti gli altri interrupt vengono disabilitati (nel registro di stato della CPU c'è un bit per abilitare/disabilitare le interruzioni) e il controllo passa automaticamente (non mediante un salto) al gestore nello spazio kernel.

Una volta terminate le operazioni di gestione il controllo ritorna al codice che era in esecuzione prima, la riabilitazione delle interruzioni e del flusso di controllo sono svolte dalla ERECT.

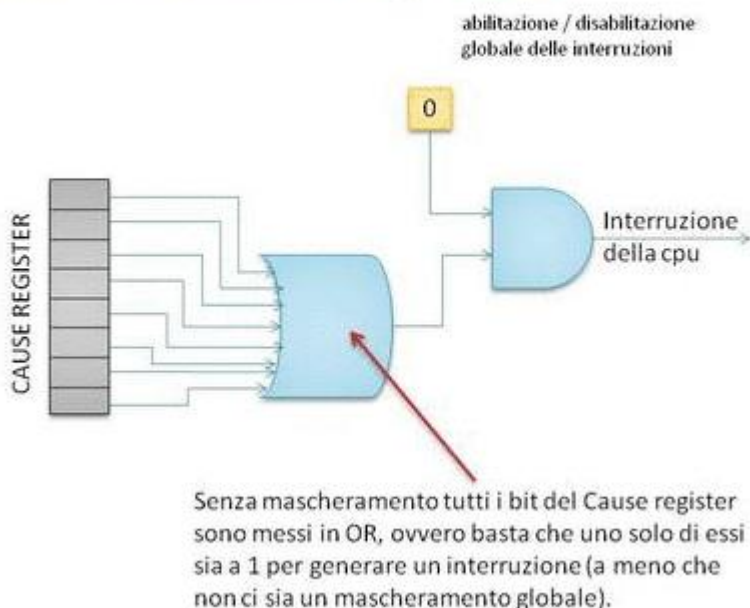
Se il tempo impiegato dal gestore è superiore alla latenza non va bene.

Un metodo per velocizzare la gestione è la vettorizzazione, la periferica invia un codice che la identifica su delle linee del bus apposite, questo codice va a finire in un registro CPU.

si conosce a prescindere l'indirizzo base del vettore, quando la CPU riceve l'interrupt utilizza il codice ricevuto come spiazzamento del vettore, quindi passa quindi ad eseguire il codice che si trova all'indirizzo Base_vettore + codice_periferica.

Il Mips32 è a gestore unico ma grazie all'uso del registro cause si velocizza la procedura.

Mascheramento globale



Anche nelle periferiche c'è un bit che abilita/dis le interruzioni.

DMA

LA gestione delle periferiche attraverso DMA permette di superare il limite della banda passante e della eccessiva latenza e permette di non eseguire busy wait.

Si usa principalmente per gestire periferiche critiche che necessitano trasferimenti di molti dati e velocemente.

Si vuole ottenere che la periferica sia autonoma nell'accesso alla memoria senza passare per la CPU.

Per utilizzare questa tecnica servono 2 ulteriori registri (**Dove** e **Quanti**) oltre a quello stato e dati, questi servono a specificare l'indirizzo di memoria al/dal quale trasferire i dati e la quantità di dati da trasferire.

Nel caso sia la periferica sia la CPU vogliano scrivere in memoria nello stesso momento ci pensa il Bus Arbitrator a mettere in attesa una delle 2.

La procedura segue **varie fasi**:

1- Predisposizione del trasferimento

I registri periferica vengono impostati dalla CPU prima del trasferimento.

2- Attivazione

La Cpu fa partire il trasferimento se il bit del registro di stato è a 1, da questo momento ci pensa la periferica, mette l'indirizzo di destinazione nel registro Dove, il dato sul bus Dati e il comando write sul bus controllo, al completamento del trasferimento di quel dato decremento Quanti e incremento Dove.

Come si accorge la CPU che la periferica ha finito? La Periferica avvisa la CPU con un segnale di interrupt

Memoria e Cache

Ci sono 2 tipi di località:

- **Località temporale** : il principio statistico è che se la locazione di un dato è stata usata allora presto, essa sarà usata ancora.
- **Località spaziale**: il principio è che se la locazione di un dato è stata usata, allora i dati adiacenti(vicini ad esso) presto tenderanno ad essere usati.

Ci avvantaggiamo del principio di località implementando la **memoria di un computer in modo gerarchico**.

La gerarchia delle memorie consiste in livelli di memoria multipli con diverse velocità e dimensione.

Le memorie più veloci sono più costose per bit rispetto alle memorie più lente e, ovviamente, sono anche più piccole.

Oggi abbiamo **3** principali tecnologie per implementare la gerarchia delle memorie:

- **La memoria principale** è implementata da **DRAM** (10 ns)
- Mentre i livelli di memoria più vicini al processore (**Caches**) usano **SRAM** (1 ns)
- **Dischi magnetici** usati per la **memoria di massa** (10 ms)

Dram è meno costosa per bit rispetto alla SRAM e meno veloce.

La differenza di prezzo nasce perché DRAM usa significativamente meno spazio per bit di memoria, e le DRAM hanno una capacità maggiore per la stessa quantità di silicio. La differenza di velocità è data da diversi fattori.

I dischi magnetici sono usati per implementare i livelli più grandi e lenti della gerarchia.

I dati sono anch'essi similmente alle memorie disposti in modo gerarchico, il livello più vicino al processore è generalmente un sotto-insieme dei livelli più lontani, e tutti i dati sono memorizzati nel livello più basso.

La gerarchia delle memorie consiste in molti livelli ma i dati sono copiati solo attraverso 2 livelli adiacente alla volta. (quindi ciò che segue riguarda un esempio con solo 2 livelli)

Block (Blocco) è la unità minima di informazione che può essere presente o non in una cache.

Se il dato richiesto dal processore si trova al più alto livello della gerarchia allora avremo una **hit**.

Se il dato non viene trovato nel livello più alto allora la richiesta si dice **miss**, allora viene effettuato l'accesso al livello più basso della gerarchia per ottenere il blocco contenente il dato.

L'**Hit rate** (x%) è la probabilità che il dato sia nel livello più alto.

Il **Miss rate** è la probabilità che il dato non sia nel livello più alto ($1 - \text{hit rate}$)

Visto che le performance sono la maggior ragione per cui usare la gerarchia delle memorie, il tempo delle hit e delle miss è importante.

Hit time è il tempo per accedere al livello più alto, che include anche il tempo necessario per capire se l'accesso è una hit o una miss.

La Miss penalty è il tempo per rimpiazzare un blocco di memoria nel livello più alto con il corrispondente blocco della memoria del livello più basso + il tempo di consegnare il blocco al processore che lo ha richiesto.

Siccome il livello più alto (si intende sempre la cache nel nostro caso) è più piccolo e costruito in modo da essere veloce, l'hit time sarà molto più basso rispetto al tempo di accesso al prossimo livello della gerarchia, che è il componente principale della miss penalty.

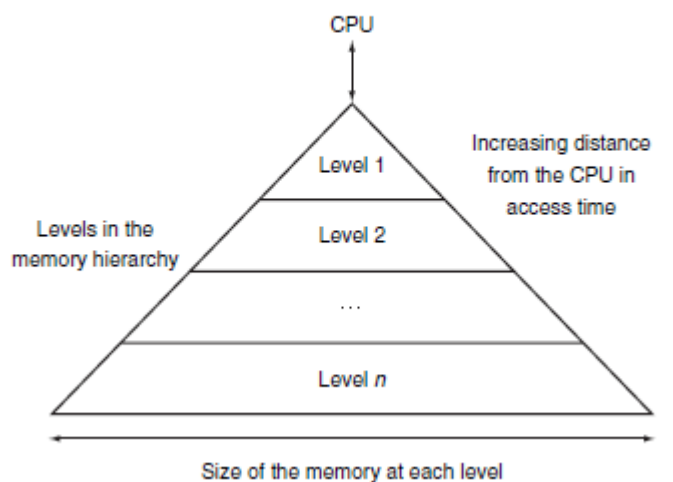
Esempio:

Hit rate= 99%

T cache = 1 ns

T memoria = 100 ns

Tempo medio= $100 \text{ ns} + 100 \text{ ns (miss penalty)} = [200 \text{ ns}] / [\text{numero accessi (100)}] = 2 \text{ ns}$



Nel caso la memoria principale risulti insufficiente per contenere i dati necessari si può utilizzare parte della memoria di massa **virtualizzandola**.

La cache consiste in un numero di blocchi, chiamiamolo N.

N è quasi sempre una qualche potenza di 2, ogni blocco contiene M Byte che è solitamente una potenza di 2 anch'essa.

Assumiamo per il nostro esempio che la nostra cache consiste in $2^7=128$ blocchi.

Ogni blocco contiene 32 byte.

Fully associative:

In uno schema fully associative, qualsiasi blocco può memorizzare il dato.

L'hardware per cercare se il dato desiderato è nella cache deve comparare i bit destinati al **tag** dell'indirizzo con i bit del tag di ogni blocco, e assicurarsi che il bit di validità sia alto.

Se c'è una miss, il passo iniziale è prendere un blocco che non è valido per inserire il dato di cui siamo in cerca.

Se non ci sono slot validi, l'hardware deve prendere un blocco da svuotare basandosi su qualche policy di sfratto.

Se il bit "dirty" del blocco è alto, allora il dato nel blocco deve essere copiato nella RAM, mentre se è 0, non c'è bisogno di ricopiarlo poiché i dati nel blocco devono essere identici ai dati nella corrispondente locazione della RAM.

Dopodiché il nuovo dato deve essere copiato dalla RAM nel blocco, impostando il bit di validità a 1 e il bit "dirty" a 0 e il tag dello slot copiato dal tag address.

Direct mapped:

Bisogna rispondere a 2 quesiti:

- 1) Come sappiamo se il dato è nella cache?
- 2) E se non è presente, come lo troviamo?

Il metodo più semplice per assegnare una locazione nella cache per ogni word in memoria è assegnare le locazioni della cache per basandosi sull'indirizzo di essa nella memoria.

Questo metodo si chiama direct mapped poiché ogni locazione di memoria è mappata direttamente verso una locazione della cache.

Ad esempio molte cache mappate usano questo metodo per trovare un blocco: (indirizzo del blocco) mod (numero di blocchi nella cache)

Ad esempio in una Cache da 8-blocchi usa i 3 bit meno significativi ($8 = 2^3$) degli indirizzi dei blocchi.

Ad esempio la figura sottostante mostra come gli indirizzi di memoria da 1(00001₂) a 29(11101₂) vengono mappati alle locazioni 1 (001₂) e 5 (101₂).

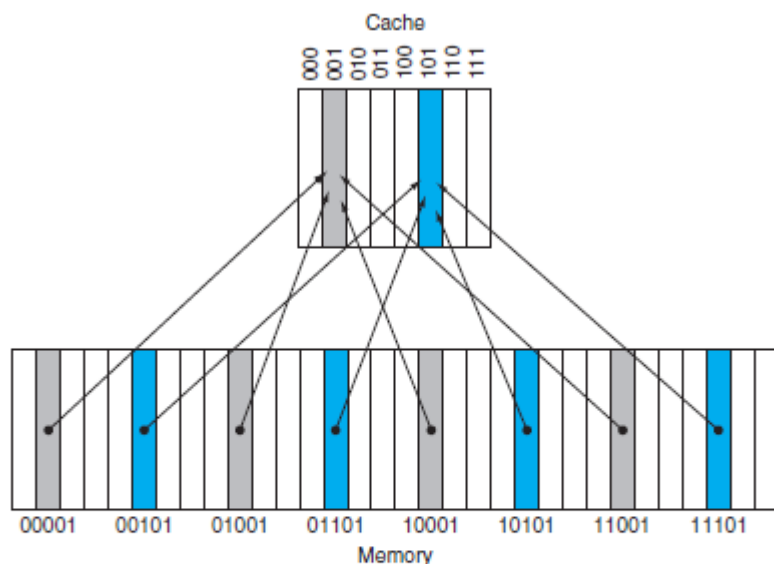


FIGURE 5.5 A direct-mapped cache with eight entries showing the addresses of memory words between 0 and 31 that map to the same cache locations. Because there are eight words in the cache, an address X maps to the direct-mapped cache word $X \bmod 8$. That is, the low-order $\log_2(8) = 3$ bits are used as the cache index. Thus, addresses 00001_{two} , 01001_{two} , 10001_{two} , and 11001_{two} all map to entry 001_{two} of the cache, while addresses 00101_{two} , 01101_{two} , 10101_{two} , and 11101_{two} all map to entry 101_{two} of the cache.

Visto che ogni locazione della cache può contenere il contenuto di un numero diverso di locazioni di memoria, come possiamo sapere dove il dato nella cache corrisponde alla word richiesta? E come sappiamo se è presente nella cache o meno? Per risolvere questo quesito aggiungiamo un set di tag nella cache.

I tag contengono l'informazione dell'indirizzo richiesto per identificare dove una word nella cache corrisponde a quella richiesta. Il tag ha bisogno di contenere la porzione superiore dell'indirizzo, corrispondente ai bit che non sono usati come indice nella cache. Per esempio nella figura sopra abbiamo bisogno dei soli 2 bit più significativi dei 5 bit dell'indirizzo di memoria nel tag, poiché i 3 bit meno significativi sono usati come indirizzo per la cache.

Aggiungiamo anche un bit di validità per segnalare se l'entry contiene o no un dato valido.

Uno svantaggio di questo metodo è che a causa di possibili conflitti dei dati ancora utili possano essere sovrascritti.

Un piccolo esempio visivo per capire meglio:

Index	V	Tag	Data
000	N		
001	N		
010	N		
011	N		
100	N		
101	N		
110	N		
111	N		

a. The initial state of the cache after power-on

Index	V	Tag	Data
000	N		
001	N		
010	Y	11 _{two}	Memory (11010 _{two})
011	N		
100	N		
101	N		
110	Y	10 _{two}	Memory (10110 _{two})
111	N		

c. After handling a miss of address (11010_{two})

Index	V	Tag	Data
000	Y	10 _{two}	Memory (10000 _{two})
001	N		
010	Y	11 _{two}	Memory (11010 _{two})
011	Y	00 _{two}	Memory (00011 _{two})
100	N		
101	N		
110	Y	10 _{two}	Memory (10110 _{two})
111	N		

e. After handling a miss of address (00011_{two})

Index	V	Tag	Data
000	N		
001	N		
010	N		
011	N		
100	N		
101	N		
110	Y	10 _{two}	Memory (10110 _{two})
111	N		

b. After handling a miss of address (10110_{two})

Index	V	Tag	Data
000	Y	10 _{two}	Memory (10000 _{two})
001	N		
010	Y	11 _{two}	Memory (11010 _{two})
011	N		
100	N		
101	N		
110	Y	10 _{two}	Memory (10110 _{two})
111	N		

d. After handling a miss of address (10000_{two})

Index	V	Tag	Data
000	Y	10 _{two}	Memory (10000 _{two})
001	N		
010	Y	10 _{two}	Memory (10010 _{two})
011	Y	00 _{two}	Memory (00011 _{two})
100	N		
101	N		
110	Y	10 _{two}	Memory (10110 _{two})
111	N		

f. After handling a miss of address (10010_{two})

FIGURE 5.6 The cache contents are shown after each reference request that *misses*, with the index and tag fields shown in binary for the sequence of addresses on page 460. The cache is initially empty, with all valid bits (V entry in cache) turned off (N). The processor requests the following addresses: 10110_{two} (miss), 11010_{two} (miss), 10110_{two} (hit), 11010_{two} (hit), 10000_{two} (miss), 00011_{two} (miss), 10000_{two} (hit), 10010_{two} (miss), and 10000_{two} (hit). The figures show the cache contents after each miss in the sequence has been handled. When address 10010_{two} (18) is referenced, the entry for address 11010_{two} (26) must be replaced, and a reference to 11010_{two} will cause a subsequent miss. The tag field will contain only the upper portion of the address. The full address of a word contained in cache block i with tag field j for this cache is $j \times 8 + i$, or equivalently the concatenation of the tag field j and the index i . For example, in cache f above, index 010_{two} has tag 10_{two} and corresponds to address 10010_{two}.

Se si scrive nella cache senza modificare il valore anche nella memoria e quindi i due valori risultano diversi, allora la cache e la memoria sono dette inconsistenti.

La via più semplice per mantenere la memoria principale e la cache consistenti è scrivere sempre il dato in entrambe le memorie, questo meccanismo è detto **write-through**.

L'alternativa al write-through è un metodo detto **write-back**, in questo schema quando avviene una scrittura, il nuovo valore viene scritto solo nel blocco della cache, il blocco modificato è scritto nel livello più basso della gerarchia quando viene rimpiazzato. Questo metodo può migliorare le performance soprattutto quando il processore può scrivere veloce o più veloce di quanto è gestibile dalla memoria principale, questo metodo è molto più complesso da implementare.

Assembly

Jal <EtichettaProcedura>

Salva nel registro \$ra(registro 31) l'indirizzo a cui tornare una volta finita la procedura. (indirizzo della jal + 4)

Jr <registro>

Salta all'indirizzo contenuto in un registro, Jr \$ra è un utilizzo tipico di jr per realizzare il ritorno da una procedura.

Convenzione programmatica:

\$a0 - \$a3 registri argomento dove passare i parametri

\$v0-\$v1 registri per la restituzione di risultati

Un parametro può essere un dato o un indirizzo

Funzioni elementari del OS chiamabili attraverso la syscall, pagina A33 appendice A, impostare nel registro \$v0 il codice della chiamata e impostare i parametri in \$a0-\$a3 come da tabella.

Syscall

Syscall essenziali:

- Exit codice 17
- readInt
- PrintInt codice 1 (passato per valore)
- readString
- PrintString (passato per indirizzo)

I registri \$t non sono salvati dalla procedura

- Devono essere salvati manualmente

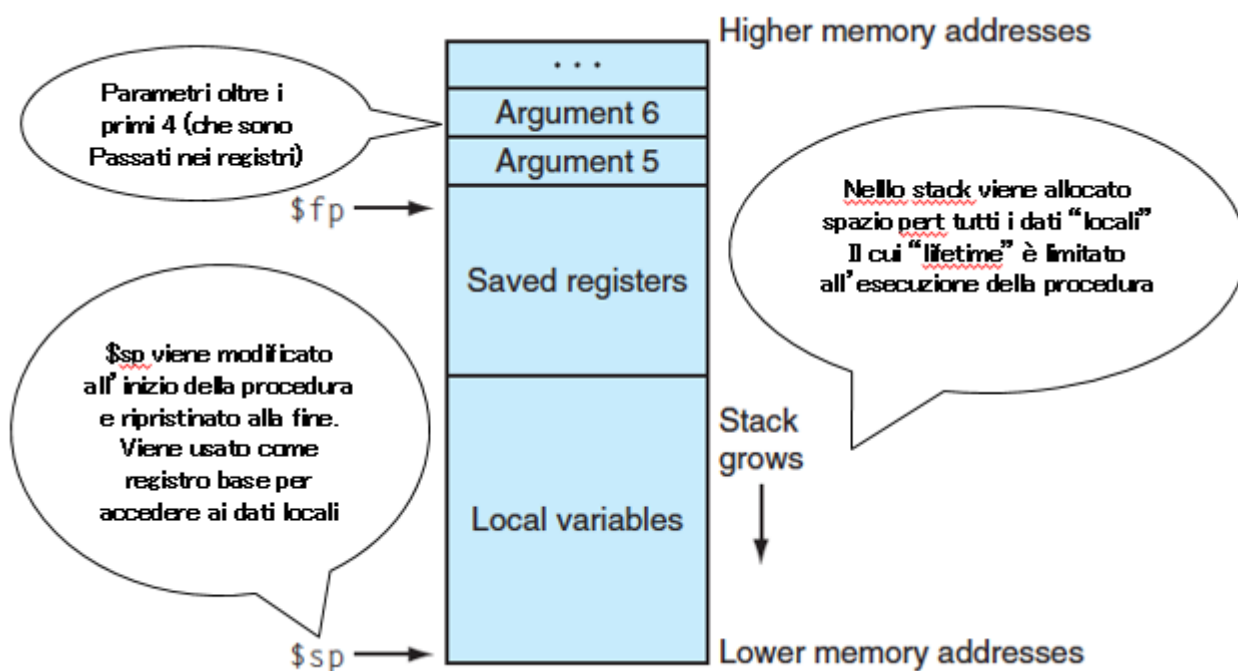
I registri \$s sono salvati dalla procedura

- Se la procedura usa i registri \$s deve salvarne il contenuto all'inizio e ripristinarlo prima del ritorno

Dove li salvo i registri \$s?

- **Stack**

Se una procedura ne chiama un'altra allora deve salvare \$ra e lo ripristini prima del ritorno



Come fa la procedura?

- Alloca spazio nello stack
 1. Decrementa \$sp per lasciare lo spazio necessario al salvataggio (1 word per ogni registro)
 2. Lo stack cresce verso il basso
 3. Salva \$ra
 4. Salva eventuali altri registri usando \$sp come registro base

5. ----
6. Ripristina i registri
7. Incrementa \$sp per riportarlo alla situazione iniziale
8. Jr \$ra ritorno alla procedura